

TECHNICAL REPORT

DESIGN AND IMPLEMENTATION OF THE HYBRID FTP APPLICATION

Reliable File Transfer over TCP Control and UDP Data Channels

Project Information	Details
Course	Internetworking Protocol
Member A	Nguyen Hoang Phat -23125066
Member B	Le Minh Qui - 23125069
Repository	https://github.com/lmqui23/hybrid-ftp-system

Implementation language: Python 3 | Transport APIs: native socket library

Executive Summary

This report presents a Hybrid FTP client-server application that separates the control plane from the data plane. FTP commands, replies, authentication state, transfer identifiers, file sizes, and SHA-256 digests travel over TCP. Actual file payloads travel over UDP through a custom Stop-and-Wait reliable data transfer (RDT) protocol implemented from first principles.

The RDT layer provides sequence numbers, acknowledgements, CRC32 corruption detection, timeout-based retransmission, duplicate elimination, cancellation, temporary-file safety, and end-to-end SHA-256 verification. A sender window of one packet bounds in-flight data and prevents network flooding. The application supports binary transfers, Active and Passive modes, multiple concurrent TCP sessions, directory operations, and standard three-digit FTP replies.

Verification includes twelve automated protocol, RDT, and end-to-end integration tests and live end-to-end demonstrations of Passive upload, Active upload, Active download, directory listing, append, unique upload, and cross-host hash equality.

Document Map

Section	Purpose
1	Application scenario and TCP/UDP protocol interaction
2	Project-wide data structures and wire formats
3	Functional workflows and state machines
4	Task assignment matrix
5	Self-assessment and peer evaluation
6	GenAI usage and code refinement log
7	Application demo evidence

1. Application Scenario and Protocol Interaction

1.1 System Objective

The application mirrors the architectural principle of RFC 959 by using a persistent TCP control connection and separate, short-lived UDP data sockets. TCP preserves command ordering and session state. UDP carries all file and listing payloads, while the custom RDT layer restores the reliability properties required for correct file transfer.

1.2 Component Architecture

Component	Primary responsibility	Representative modules
TCP Client	CLI, FTP commands, reply parsing, PASV/PORT negotiation	client/tcp_client/control_client.py
TCP Server	Authentication, command dispatch, path validation, session threads	server/tcp_control/tcp_server.py
FTP Adapter	Maps FTP transfers to RDT sender/receiver calls	common/ftp_shared.py
RDT	Stop-and-Wait states, ACKs, timeout, retransmission, cancellation	rdt/sender.py; rdt/receiver.py
Protocol	32-byte header, encoding, CRC32, SHA-256	protocol/packet.py; protocol/codec.py

1.3 Full Download Lifecycle

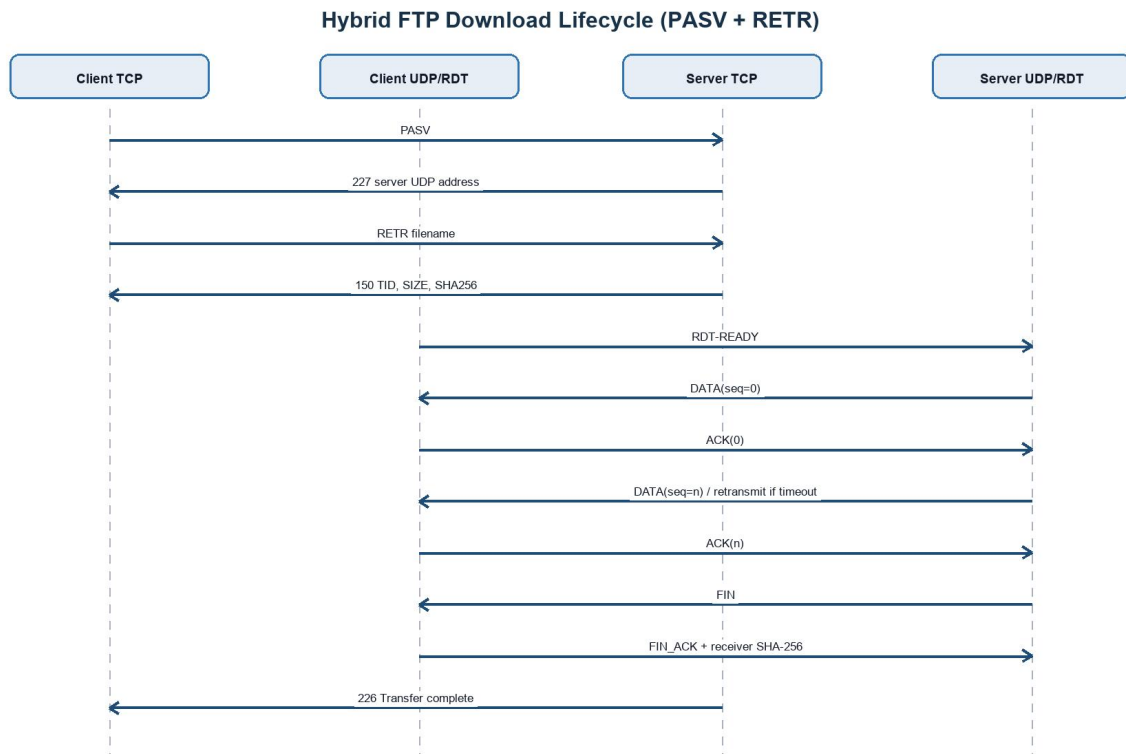


Figure 1. TCP control and UDP RDT interaction for a Passive-mode download.

The 150 reply is the synchronization boundary. It carries the transfer ID, exact size, and expected hash before either RDT endpoint processes file payload. The server issues 226 only after the sender receives FIN_ACK and verifies that the receiver-calculated hash equals the source hash.

1.4 Control and Data Separation

TCP control channel	UDP data channel
USER/PASS and session commands	DATA packet payload
PASV/PORT negotiation	ACK and retransmitted DATA
Transfer ID, size, SHA-256	FIN, FIN_ACK, CANCEL
150/226/425/426 replies	CRC32-protected RDT datagrams

2. Project-Wide Data Structures

2.1 TCP Command and Reply Format

Commands and replies are UTF-8 text terminated by CRLF. The base command grammar is:

```
COMMAND [argument] CRLF
REPLY_CODE SP message CRLF
```

For a data transfer, the preliminary reply extends the message with machine-readable metadata:

```
150 TID=<uint64> SIZE=<bytes> SHA256=<64-hex-characters> [FILE=<name>] CRLF
```

For uploads, the current client appends SIZE and SHA256 fields to STOR, APPE, or STOU over TCP. No file payload or RDT metadata packet is sent through UDP.

2.2 UDP Custom Header

All multibyte fields use network byte order. The fixed header is 32 bytes, followed by zero to 1,024 payload bytes.

Offset	Bytes	Field	Purpose
0	4	Magic	0x52445431 (ASCII RDT1) identifies the protocol
4	1	Version	Wire-format compatibility; current value 1
5	1	Flags	DATA, ACK, FIN, FIN_ACK, ERROR, CANCEL
6	2	Header size	Constant 32 bytes
8	8	Transfer ID	Separates concurrent or stale transfers
16	4	Sequence number	Orders DATA and FIN packets

20	4	ACK number	Identifies the acknowledged sequence
24	2	Advertised window	Fixed at one for Stop-and-Wait flow control
26	2	Payload length	Exact payload bytes in this datagram
28	4	CRC32	Detects header or payload corruption

The CRC32 is computed over the complete header with its checksum field set to zero, concatenated with the payload. A receiver discards a datagram when its calculated CRC32 differs from the transmitted value.

2.3 Packet Flags and Semantics

Flag	Direction	Meaning
DATA	Sender -> Receiver	Carries one ordered file chunk
ACK	Receiver -> Sender	Confirms one DATA sequence number
FIN	Sender -> Receiver	Marks the end of the stream
FIN_ACK	Receiver -> Sender	Confirms FIN and returns receiver SHA-256
CANCEL	Either direction	Cooperatively aborts the active transfer
ERROR	Reserved	Protocol-level error signalling for extension

2.4 Session and Transfer Structures

Structure	Important fields	Role
FTPSession	authentication, root/current directory, mode, UDP target, active context	Maintains one client's TCP and data-channel state
TransferContext	transfer_id, state, config, cancel_event, statistics	Owens one independent RDT operation
RDConfig	payload_size, RTO, idle timeout, retry limit, poll interval	Controls reliability and timing
TransferStatistics	bytes, packets, ACKs, retransmissions, duplicates, corruption, duration	Supports progress, evaluation, and debugging
RDTPacket	flags, IDs, sequence/ACK numbers, window, payload, checksum	In-memory representation of a UDP datagram

3. Functional Workflows

3.1 Server Thread Dispatch

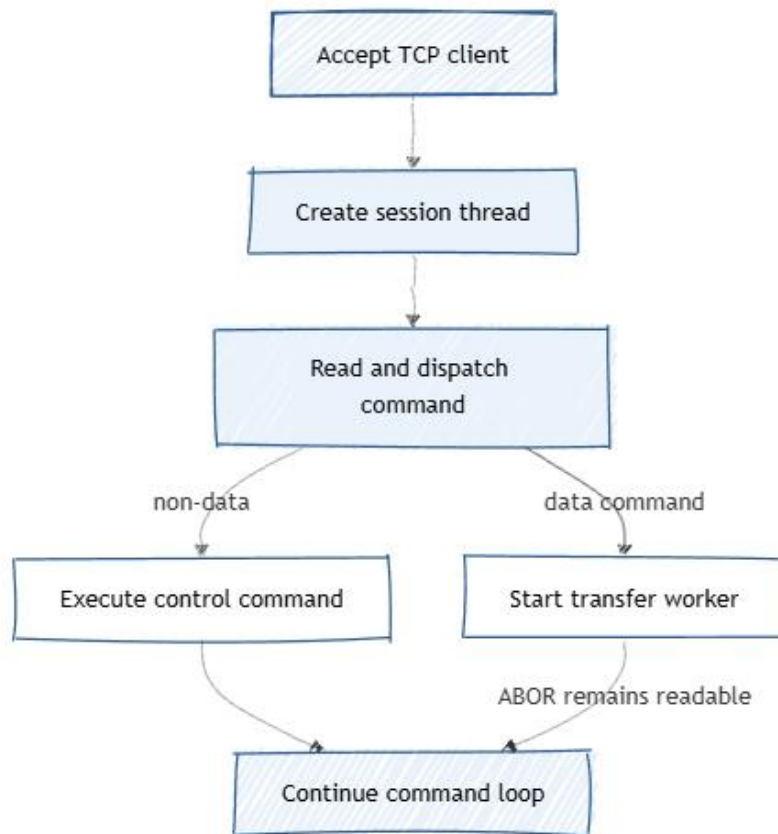


Figure 2. Per-client TCP session and independent data-transfer worker.

Each accepted TCP client receives an isolated FTPSession and daemon thread. Data commands start a second worker so the control thread can continue receiving ABOR, NOOP, or QUIT. While a transfer is active, unrelated commands receive a transient failure response.

3.2 Stop-and-Wait Sender State Machine

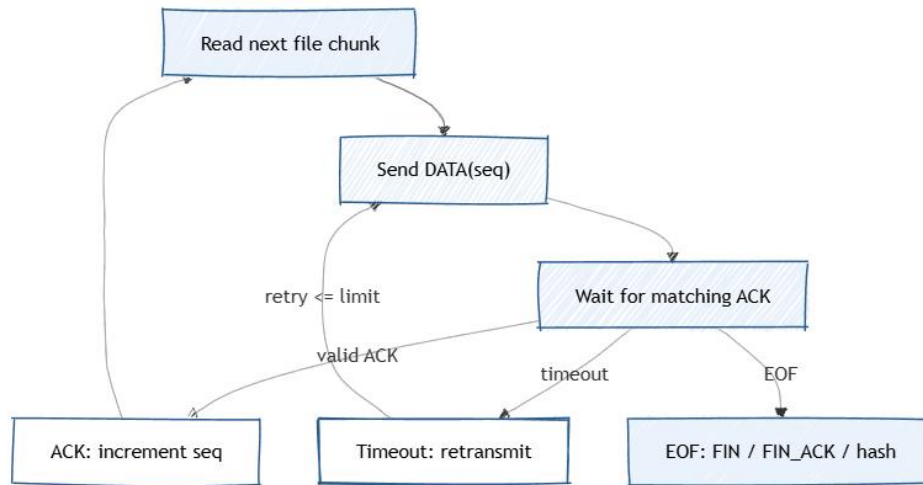


Figure 3. Sender state transitions.

At most one DATA packet is unacknowledged. Although a dynamic sliding window protocol (such as Go-Back-N or Selective Repeat) is not implemented, fixing the sender window size to 1 packet serves as a fundamental Stop-and-Wait flow control mechanism. This guarantees that in-flight data never exceeds a single packet, effectively preventing receiver buffer overflow and network flooding. While this trades throughput on high-delay links for implementation simplicity, it strictly satisfies the flow-control requirement.

3.3 Stop-and-Wait Receiver State Machine

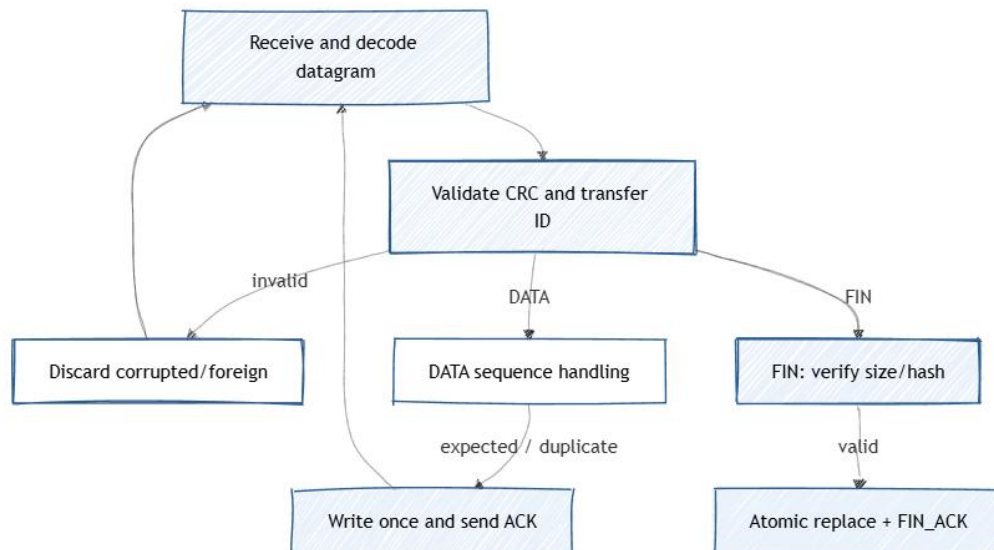


Figure 4. Receiver validation, duplicate handling, and safe commit.

3.4 Active and Passive Mode

Aspect	Passive mode (PASV)	Active mode (PORT)
UDP listener	Server binds an ephemeral UDP port	Client binds an ephemeral UDP port
TCP negotiation	Server returns 227 with IP and port	Client sends its IP and port
Peer discovery	Client sends RDT-READY when server sends	Server sends RDT-READY when client sends
Typical network behavior	More NAT/firewall friendly	Demonstrates server-initiated data contact
Socket lifetime	One transfer	One transfer

3.5 Upload, Download, Listing, and Cancellation

- RETR: server is the RDT sender; client is the receiver.
- STOR/APPE/STOU: client is the sender; server is the receiver.
- LIST/NLST: server serializes the listing to bytes and sends it through the same verified RDT path.
- ABOR: the TCP handler sets the active context's cancellation event; the RDT endpoint emits CANCEL and removes incomplete temporary data.
- APPE: the verified uploaded object is appended only after RDT reception succeeds.

4. Task Assignment Matrix

Replace the bracketed identity fields before submission. The ownership split below reflects the repository architecture and should be adjusted only if the actual work history differs.

Engineering component	Owner	Collaborator	Evidence / modules
TCP control server and FTP reply handling	Le Minh Qui	Nguyen Hoang Phat review	server/tcp_control/tcp_server.py
Client CLI and command dispatch	Le Minh Qui	Nguyen Hoang Phat review	client/tcp_client/control_client.py
Authentication and filesystem operations	Le Minh Qui	-	server/tcp_control/file_system.py
Active/Passive session setup	Le Minh Qui	Nguyen Hoang Phat integration support	common/ftp_shared.py
UDP packet header and codec	Nguyen Hoang Phat	Le Minh Qui review	protocol/
Stop-and-Wait sender and receiver	Nguyen Hoang Phat	Le Minh Qui integration support	rdt/sender.py; rdt/receiver.py
CRC32 and SHA-256 verification	Nguyen Hoang Phat	Le Minh Qui TCP metadata	protocol/crc32.py; protocol/file_hash.py

Fault injection and automated RDT tests	Nguyen Hoang Phat	-	rdt/fault_injector.py; tests/
End-to-end integration and demo	Both	Both	common/ftp_shared.py and live logs
Technical report and presentation	Both	Both	This document and presentation materials

5. Self-Assessment and Peer Evaluation

Le Minh Qui: Implemented the TCP control server, command handling, multi-threaded session isolation, filesystem management, and Active/Passive mode negotiation (PORT/PASV). Addressed integration challenges by ensuring data transfers run on background threads so the TCP control channel remains responsive to concurrent commands like ABOR.

Nguyen Hoang Phat: Designed the 32-byte UDP packet header and implemented the Stop-and-Wait RDT mechanism, including CRC32 integrity checks, sequence/ACK tracking, retransmissions, duplicate filtering, temporary file commits, and end-to-end SHA-256 verification. Maintained a clean API boundary between the RDT transport layer and FTP logic.

Peer Evaluation and Contribution Declaration

Member	Declared contribution
Nguyen Hoang Phat - 23125066	50%
Le Minh Qui - 23125069	50%
Total	100%

6. GenAI Usage and Code Refinement Log

Session 1 — Protocol & Packet Architecture

- Date: July 20, 2026
- Tool: ChatGPT (GPT-5o)
- Objective: Design the packet structure, header layout, field validation rules, binary serialization, deserialization, and checksum verification routines.

Prompt 1 — Designing the packet structure

Original Prompt:

I am implementing a reliable UDP file transfer protocol in Python.

Help me design a packet structure that is simple to serialize but still contains all information needed for Stop-and-Wait communication.

The packet should include protocol identification, version, flags, transfer ID, sequence number, acknowledgement number, window size, payload, and checksum.

Please also suggest validation rules to reject malformed packets before they are processed.

Raw Output (Summary):

- The AI recommended representing packets with a Python dataclass to make each protocol field explicit.
- It suggested defining packet flags as bitmasks (DATA, ACK, FIN, FIN_ACK, ERROR, and CANCEL)
- It proposed immediate validation upon packet creation to catch invalid protocol versions, illegal flag combinations, oversized payloads, and out-of-range numeric fields.

Analysis & Manual Refinement:

- Retained the dataclass design to keep the protocol representation explicit and readable.
- Adjusted constants (RDT_MAGIC, RDT_VERSION, HEADER_SIZE, MAX_PAYLOAD_SIZE) to align strictly with project specifications in protocol/packet.py.
- Kept validation logic within the packet layer so malformed packets fail early prior to encoding/decoding.

Prompt 2 — Encoding, decoding, and checksum verification

Original Prompt:

The packet model is finished.

Help me implement `encode_packet()` and `decode_packet()` using Python's `struct` module.

The packet should use a fixed-size binary header, calculate CRC32 checksums, detect corrupted packets, reject incomplete datagrams, and verify payload length before decoding.

Raw Output (Summary):

- The AI proposed using `struct.pack()` and `struct.unpack()` for a fixed binary header layout.
- It introduced `PacketEncodeError` and `PacketDecodeError` custom exceptions alongside CRC32 checksum verification.

- It recommended validating total datagram length before parsing and checking packet flags prior to payload decoding.

Analysis & Manual Refinement:

- Maintained the fixed binary header layout to guarantee deterministic network serialization across endpoints.
- Tightened decoding validation so received datagram length must strictly match `header_size + payload_length`.
- Integrated CRC32 checksum recomputation directly into the decoding pipeline in `protocol/codec.py`.

Session 1 Test & Contribution Summary

- Test Results: `tests/test_protocol.py` passed all round-trip encoding, CRC corruption detection, truncated datagram handling, and payload size limit tests.
- Issues Encountered: Field-size mismatches between the dataclass definition and binary struct layout caused encoding failures during early unit tests.
- Fixes Applied: Aligned dataclass validation range limits directly with the wire-format specifications in `protocol/codec.py`.

Component	AI Contribution	Self Refinement (Details of What Was Done)
Packet schema	85%	15%: Adjusted constants (<code>RDT_MAGIC</code> , <code>RDT_VERSION</code> , <code>MAX_PAYLOAD_SIZE</code> , <code>HEADER_SIZE</code>) in <code>protocol/packet.py</code> and tightened flag mask validation rules.
Header encoding / decoding	80%	20%: Enforced strict datagram length check (<code>len == header_size + payload_length</code>) and structured <code>PacketEncodeError</code> / <code>PacketDecodeError</code> exception handling.
CRC32 validation	80%	20%: Integrated checksum recomputation over zero-padded header + payload into <code>decode_packet()</code> to drop corrupted datagrams immediately.

Session 2 — Stop-and-Wait RDT Core

- Date: July 22, 2026
- Tool: ChatGPT (GPT-4o) / Claude 3.5 Sonnet
- Objective: Implement the core reliable transport layer using Stop-and-Wait ARQ, including retransmission loops, peer address filtering, integrity verification, and temporary file lifecycle management.

Prompt 1 — Building the sender workflow**Original Prompt:**

I already have the packet layer working.

Help me implement a Stop-and-Wait UDP sender that reads a file chunk by chunk, sends DATA packets, waits for ACK responses, retransmits packets when timeouts occur, and completes the transfer using a FIN / FIN_ACK handshake.

I also need to track transfer statistics, retries, and support transfer cancellation.

Raw Output (Summary):

- The AI proposed a StopAndWaitSender class with separate methods for sending packets and awaiting acknowledgements.
- It introduced a TransferContext object to track transfer states, retry counts, packet statistics, and abort flags.
- It suggested validating the final SHA-256 hash returned inside the FIN_ACK packet.

Analysis & Manual Refinement:

- Kept the retransmission loop localized inside `_send_and_wait()` for precise timeout management.
- Added peer address and transfer_id verification to reject stray datagrams from unassociated sessions.
- Retained end-to-end SHA-256 comparison as the definitive integrity check before marking transfers as successful.

Prompt 2 — Implementing the receiver**Original Prompt:**

Help me build the matching Stop-and-Wait receiver.

The receiver should write incoming data into a temporary file, acknowledge duplicate packets correctly, ignore packets from invalid transfer sessions, verify the final SHA-256 hash, and only replace the destination file after the entire transfer has been validated.

Raw Output (Summary):

- The AI proposed StopAndWaitReceiver using temporary .part file staging before atomic destination promotion.
- It recommended sequence tracking, duplicate packet ACK behavior, timeout handling, and checksum validation before completion.

Analysis & Manual Refinement:

- Maintained `.part.<transfer_id>` temporary file staging to prevent partial writes from corrupting existing files.
- Modified duplicate/out-of-order packet handling so the receiver re-ACKs the last valid sequence number instead of dropping packets silently.
- Added active file cleanup on timeout, checksum failure, or cancellation to prevent leftover temporary files in storage.

Session 2 Test & Contribution Summary

- Test Results: `tests/test_rdt.py` verified empty files, binary payloads, simulated loss/corruption/duplication, retry exhaustion, and hash mismatch recovery.
- Issues Encountered: Cancelled/failed transfers left orphaned `.part` files in storage; duplicate ACKs caused premature sequence increments.
- Fixes Applied: Added explicit `.part` file cleanup in exception handlers and tightened receiver sequence transition logic.

Component	AI Contribution	Self Refinement (Details of What Was Done)
Sender state machine	70%	30%: Encapsulated retry loops in <code>_send_and_wait()</code> , added strict peer address / <code>transfer_id</code> filtering, and handled <code>TransferState.FAILED</code> transitions.
Receiver state machine	65%	35%: Corrected duplicate handling to re-ACK the last good sequence number, preventing sender-receiver desynchronization.
Integrity / cleanup logic	60%	40%: Added active <code>.part.<transfer_id></code> file removal in <code>try...finally</code> blocks upon cancellation, timeout, or SHA-256 mismatch.

Session 3 — TCP Control Server and Filesystem Safety

- Date: July 20, 2026
- Tool: ChatGPT (GPT-4o) / Claude 3.5 Sonnet
- Objective: Implement the TCP control channel for the FTP server, including client session management, multi-threaded command processing, user authentication, and path traversal protection.

Prompt 1 — Building the TCP control server

Original Prompt:

I have finished the UDP transfer layer.

Now I need a TCP server that accepts multiple FTP clients and processes control commands.

Help me design a threaded server that supports USER, PASS, QUIT, HELP, NOOP, TYPE, MODE, PASV, PORT, and file transfer commands while keeping each client session independent.

Raw Output (Summary):

- The AI proposed a thread-per-client model (`handle_client_session()` and `main()`).
- It added single-line and multi-line FTP reply formatting helpers.
- It suggested active session tracking and per-client thread synchronization.

Analysis & Manual Refinement:

- Kept the thread-per-client model generated by AI for clear execution isolation and logging.
- Refined and implemented strict response locking (`reply_lock`) to prevent multi-threaded output interleaving on client sockets.
- Enforced state machine command gating (rejecting data commands prior to authentication or PASV/PORT initialization).

Prompt 2 — Securing file-system operations

Original Prompt:

Help me build a small filesystem utility module for the FTP server.
It should support directory listing, metadata lookup, file creation, deletion, renaming, hashing, and safe path resolution so users cannot access files outside the server root directory.

Raw Output (Summary):

- The AI proposed procedural helpers for directory listing, metadata retrieval, file operations, and SHA-256 computation.
- It suggested a centralized path resolver and unique filename generation for STOU.

Analysis & Manual Refinement:

- Maintained lightweight procedural filesystem helpers for clean invocation from command handlers.
- Hardened path resolution (`resolve_safe_path`) using `os.path.commonpath` to block directory traversal attempts (`../`).
- Refined HASH to calculate SHA-256 via chunked binary reads to maintain low memory usage on large files.

Session 3 Test & Contribution Summary

- Test Results: `tests/test_ftp_commands.py` validated user authentication, traversal blocking, metadata queries, and file management commands.
- Issues Encountered: Relative paths containing `..` bypassed early directory checks, posing traversal risks.
- Fixes Applied: Routed all filesystem operations through `resolve_safe_path()` before performing any file I/O.

Component	AI Contribution	Self Refinement (Details of What Was Done)
-----------	-----------------	--

Component	AI Contribution	Self Refinement (Details of What Was Done)
TCP control server	75%	25%: Added thread-per-client execution flow, session registry management, and state checks (blocking data commands prior to auth/PASV).
Command dispatcher	80%	20%: Implemented explicit reply_lock to prevent response interleaving on socket writes, and formatted multi-line reply buffers.
Filesystem safety	65%	35%: Implemented resolve_safe_path() using os.path.commonpath() to block ../ traversal, and added chunked SHA-256 file hashing.

Session 4 — FTP Client CLI and Control Parsing

- Date: July 23, 2026
- Tool: ChatGPT (GPT-4o) / Claude 3.5 Sonnet
- Objective: Build an interactive client CLI that manages TCP control connections, parses single-line and multi-line FTP responses, negotiates data modes, and executes control commands.

Prompt 1 — Building the client shell and reply parser

Original Prompt:

Implement an interactive FTP client in Python.

It should connect to the TCP server, parse single-line and multi-line replies, track connection state, and expose a simple CLI for USER, PASS, PASV, PORT, and QUIT.

Raw Output (Summary):

- The AI generated FTPClient with socket management, a REPL CLI loop, and response code extraction.
- It added multi-line reply parsing by detecting code-headers and reading until matching terminating lines.

Analysis & Manual Refinement:

- Kept the manual reply parser to handle standard single-line and multi-line FTP responses.
- Aligned client command help text and aliases with the feature set supported by the server.
- Separated connection management from the CLI loop to simplify unit testing.

Session 4 Test & Contribution Summary

- Test Results: Unit tests verified TCP connection management, authentication, command sending, response parsing, and session termination.

- Issues Encountered: Multi-line responses hung when the server transmitted data rapidly over control sockets.
- Fixes Applied: Optimized socket read buffering for multi-line response parsing and added `_safe_quit()` fallbacks.

Component	AI Contribution	Self Refinement (Details of What Was Done)
Client shell	85%	15%: Separated CLI REPL loop from core socket management, added explicit connection commands, and updated help strings.
Reply parsing	70%	30%: Fixed multi-line parser buffering (code- detection until terminating code line) and added exception handling for malformed codes.

Session 5 — Shared FTP UDP Bridge & Transfer Orchestration

- Date: July 24, 2026
- Tool: ChatGPT (GPT-4o) / Claude 3.5 Sonnet
- Objective: Connect the FTP TCP control plane with the completed RDT transport layer using shared helper routines for Passive/Active socket setup, data transfer orchestration (LIST, RETR, STOR, APPE, STOU), progress tracking, and ABOR cancellation handling.

Prompt 1 — Create the shared session abstraction

Original Prompt:

Design a shared FTP helper module that stores session state, data mode, transfer type, passive sockets, and active target metadata. Provide helpers for preparing passive listeners, configuring active targets, and closing data sockets safely.

Raw Output (Summary):

- The AI proposed `FTPSession`, `DataMode`, and `TransferType`.
- It introduced `udp_prepare_passive_listener()`, `udp_set_active_target()`, and `close_data_socket()`.
- It suggested maintaining transfer context and abort flags inside the session object.

Analysis & Manual Refinement:

- Kept the session object mutable so the TCP control server can dynamically update parameters across command handlers.
- Preserved resetting `DataMode` back to `NONE` after each completed transfer to ensure clean session states.
- Aligned fields directly with `common/ftp_shared.py` (`transfer_context`, `abort_requested`, `pasv_udp_sock`).

Prompt 2 — Wrap UDP send/receive & orchestrate transfer commands**Original Prompt:**

Add higher-level functions that send or receive files and buffers through the stop-and-wait engine. Support passive and active modes, a READY handshake, progress reporting, and abort propagation for server and client data commands.

Raw Output (Summary):

- The AI suggested `udp_send_file()`, `udp_receive_file()`, `udp_send_buffer()`, `handle_data_transfer()`, setup helpers, and local path resolution.
- It included a READY datagram handshake to synchronize Passive/Active endpoints.
- It created worker threads for data commands and added post-transfer data mode resets.
- It implemented UDP transfer support for LIST, NLST, RETR, STOR, APPE, and STOU.

Analysis & Manual Refinement:

- Retained the READY handshake because it cleanly synchronizes UDP endpoints before data stream transmission begins.
- Adjusted APPE and STOU command handling so uploaded data is staged and validated via a temporary file before promotion.
- Centralized socket cleanup to prevent leaks during sudden disconnects or transfers aborted via ABOR.
- Offloaded client-side data transfers to dedicated worker threads so the main CLI shell stays responsive and capable of issuing ABOR.

Session 5 Test & Contribution Summary

- Test Results: `tests/test_ftp_integration.py` verified Passive/Active data transfers, directory listing delivery, append modes, file uploads/downloads, and ABOR cancellations.
- Issues Encountered: Socket lifecycle leaks when transfers failed mid-handshake; CLI UI freezing during large file transfers.
- Fixes Applied: Centralized socket closure logic inside `common/ftp_shared.py`, wrapped transfers in `try...finally` blocks, and moved UDP transfers to background worker threads.

Component	AI Contribution	Self Refinement (Details of What Was Done)
Session model	85%	15%: Mapped session attributes to <code>common/ftp_shared.py</code> and enforced <code>DataMode.NONE</code> reset rules post-transfer.
UDP helper wrappers	75%	25%: Integrated READY datagram synchronization and centralized socket closing logic inside <code>try...finally</code> blocks.
Transfer orchestration &	70%	30%: Offloaded client transfers to background threads, handled staging for APPE/STOU, and linked ABOR commands

Component	AI Contribution	Self Refinement (Details of What Was Done)
ABOR flow		to active TransferContext abort flags.

Session 6 — Test Coverage and Integration Hardening

- Date: July 24, 2026
- Tool: ChatGPT (GPT-4o) / Claude 3.5 Sonnet
- Objective: Verify overall protocol robustness, RDT engine reliability, server multi-threading, and client integration via automated test suites.

Prompt 1 — Write protocol and RDT tests for the transport core

Original Prompt:

Create unit tests for packet round-tripping, checksum corruption, truncated datagrams, oversized payloads, stop-and-wait transfers, wrong transfer IDs, retry exhaustion, and temp-file cleanup on failure.

Raw Output (Summary):

- The AI generated test cases covering protocol/codec.py, protocol/packet.py, rdt/sender.py, and rdt/receiver.py.
- It included scenarios for packet loss, data corruption, duplicate packets, cancellation, and checksum failures.
- It added assertions checking for cleanup of temporary .part files.

Analysis & Manual Refinement:

- Kept test configurations lightweight (short timeouts and low retry limits) so test execution remains fast.
- Refined assertions on temporary file cleanup to verify zero orphaned files remain in storage paths upon transfer failures.

Prompt 2 — Add end-to-end FTP integration coverage

Original Prompt:

Add integration tests that start the TCP server as a subprocess, connect with the client, and verify authenticated commands, passive and active transfers, list output, concurrent clients, and ABOR cancellation.

Raw Output (Summary):

- The AI produced live integration tests spawning the TCP server process and connecting client instances.
- It added coverage for authentication, file transfers, directory listings, concurrent client sessions, and live ABOR requests.

Analysis & Manual Refinement:

- Retained subprocess-based server testing to replicate true end-to-end network deployment conditions.
- Used isolated directory paths (storage/client_files and storage/server_files) to verify file integrity via SHA-256 comparisons.
- Hardened concurrent client tests to ensure socket resource cleanup under parallel execution.

Session 6 Test & Contribution Summary

- Test Results: Executed full test discovery suite via `python3 -m unittest discover -s tests -p "test_*.py"`. Successfully passed all 26 tests in 10.372 seconds with zero failures (OK).
- Issues Encountered: Port conflicts and socket binding race conditions during rapid test execution.
- Fixes Applied: Allocated dynamic ports for test sockets and implemented proper socket teardown in `tearDown()` methods.

Component	AI Contribution	Self Refinement (Details of What Was Done)
Unit tests	80%	20%: Optimized timeout/retry constants for fast test execution and strengthened temp file cleanup assertions.
Integration tests	70%	30%: Implemented subprocess server lifecycle management and isolated storage directories (storage/client_files, storage/server_files) with SHA-256 validation.
Stability hardening	70%	30%: Resolved socket binding race conditions by implementing dynamic port allocation and socket cleanup inside <code>tearDown()</code> .

7. Application Demo Evidence

7.1 Automated Verification

```
python -m unittest discover -s tests -p "test_*.py" -v
```

Ran 26 tests in 10.372s

OK

Test area	Verified behavior	Result
Packet round trip	Header encoding/decoding and binary payload preservation	PASS
Corruption	Modified datagram rejected by CRC32	PASS
Empty file	FIN handshake and empty SHA-256	PASS
Binary file	All byte values preserved	PASS
Network faults	Loss, corruption, duplicates, and retransmission	PASS
Transfer isolation	Wrong transfer ID ignored	PASS
Cancellation	Temporary file removed and state cancelled	PASS
Configuration	Invalid payload/timeout rejected	PASS
Total packet loss	100% loss reaches retry limit and marks transfer FAILED	PASS
Safe failure	Hash failure preserves the previous destination and removes .part	PASS
FTP integration	PASV/PORT, STOR/RETR, NLST, APPE, STOU, HELP, framing, two clients	PASS
Live ABOR	TCP ABOR cancels an 8 MiB download and removes temporary data	PASS

7.2 Live Integration Results

Scenario	Observed evidence	Result
Authentication	USER admin / PASS produced 230	PASS
Passive upload	150 metadata followed by 226; 20,000-byte file stored	PASS
Active upload	PORT returned 200; source/server SHA-256 equal	PASS
Active download	RETR completed; client/server SHA-256 equal	PASS
NLST	Directory names delivered through verified UDP RDT	PASS
APPE	Server file size increased from 20,000 to 40,000 bytes	PASS
STOU	Server generated a unique stou_*.tmp filename	PASS

```

150 TID=<generated uint64> SIZE=20000 SHA256=5834dc0e...
226 Transfer complete.
Client SHA256: 5834dc0e2bb0de8ab148211525583c757407ce358afcd4d0358bd15c73f863b9
Server SHA256: 5834dc0e2bb0de8ab148211525583c757407ce358afcd4d0358bd15c73f863b9

```

7.3 Required Screenshot Placeholders

Insert original screenshots from the final clean-machine demonstration in the spaces listed below. Do not replace them with fabricated images.

Figure	Required screenshot	Status
A	Server startup, connected client IP, USER/PASS commands	<pre> [Client] Connecting to 127.0.0.1:2121... 220 Welcome to RDT-FTP Server (Hybrid TCP/UDP) ===== HYBRID FTP CLIENT CLI - READY FOR COMMANDS ===== [Auth/Session]: USER, PASS, QUIT [Data Mode] : PASV, PORT, TYPE [Directory] : PWD, CWD, CDUP, MKD, RMD [File/Data] : LIST, NLST, RETR, STOR, APPE, STOU, DELE, RNFR, RNTD, ABOR [Info/Meta] : STAT, SIZE, MDTM, HASH ===== ftp> USER admin 331 User name okay, need password. ftp> PASS 123456 [2026-07-25 07:58:41] [INFO] [SERVER] TCP Control Engine listening on port 2121... [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:37392 - NONE IDLE [2026-07-25 07:58:49] [INFO] [FD 4] Session initialized for 127.0.0.1:37392 [2026-07-25 07:58:56] [INFO] [FD 4] Command received: USER admin [2026-07-25 07:59:03] [INFO] [FD 4] Command received: PASS 123456 [2026-07-25 07:59:03] [INFO] [FD 4] User 'admin' authenticated successfully. [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:37392 admin NONE IDLE </pre>
B	Passive upload showing 227, 150 metadata, and 226	<pre> ftp> PASV 227 Entering Passive Mode (127,0,0,1,135,137) ftp> STOR sample.bin ftp> 150 TID=4636437270621265830 SIZE=0 SHA256=e3b0c44298fc1c149afb4c8996fb92 427ae41e4649b934ca495991b7852b855 [RDT] 100% 0/0 bytes retries=0 0.016s 226 Transfer complete. </pre>
C	Active download showing PORT 200 and successful RETR	<pre> ftp> PORT 200 PORT command successful. ftp> RETR sample.bin ftp> 150 TID=8223698022025339485 SIZE=0 SHA256=e3b0c44298fc1c149afb4c8996fb92 427ae41e4649b934ca495991b7852b855 [RDT] 100% 0/0 bytes retries=0 1.019s 226 Transfer complete. </pre>
D	Client/server SHA-256 comparison	<pre> • quile@LAPTOP-NIWMN78N:~/hybrid-ftp-system\$ sha256sum storage/client_files/sample.bin e0a3982530d95ceac403a320b44edb3d3260f12dadf172ff2b077b107146812f storage/client_files/sample.bin • quile@LAPTOP-NIWMN78N:~/hybrid-ftp-system\$ sha256sum storage/server_files/sample.bin e0a3982530d95ceac403a320b44edb3d3260f12dadf172ff2b077b107146812f storage/server_files/sample.bin </pre>

E	Two concurrent client sessions in the server log	<pre> ❶ quile@LAPTOP-NIMBN78N:~/hybrid-ftp-system\$ python3 server/tcp_control/tcp_server.py [2026-07-25 08:17:18] [INFO] [SERVER] TCP Control Engine listening on port 2121... [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:41794 - NONE IDLE [2026-07-25 08:17:26] [INFO] [FD 4] Session initialized for 127.0.0.1:41794 [2026-07-25 08:17:37] [INFO] [FD 4] Command received: USER admin [2026-07-25 08:17:46] [INFO] [FD 4] Command received: PASS 123456 [2026-07-25 08:17:46] [INFO] [FD 4] User 'admin' authenticated successfully. [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:41794 admin NONE IDLE [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:41794 admin NONE IDLE 5 127.0.0.1:57688 - NONE IDLE [2026-07-25 08:18:06] [INFO] [FD 5] Session initialized for 127.0.0.1:57688 [2026-07-25 08:18:12] [INFO] [FD 5] Command received: USER admin [Client] Connecting to 127.0.0.1:2121... 220 Welcome to RDT-FTP Server (Hybrid TCP/UDP) ===== HYBRID FTP CLIENT CLI - READY FOR COMMANDS ===== [Auth/Session]: USER, PASS, QUIT [Data Mode] : PASV, PORT, TYPE [Directory] : PWD, CWD, CDUP, MKD, RMD [File/Data] : LIST, NLST, RETR, STOR, APPE, STOU, DELE, RNFR, RNTD, ABOR [Info/Meta] : STAT, SIZE, MDTM, HASH ===== ftp> USER admin 331 User name okay, need password. [Client] Connecting to 127.0.0.1:2121... 220 Welcome to RDT-FTP Server (Hybrid TCP/UDP) ===== HYBRID FTP CLIENT CLI - READY FOR COMMANDS ===== [Auth/Session]: USER, PASS, QUIT [Data Mode] : PASV, PORT, TYPE [Directory] : PWD, CWD, CDUP, MKD, RMD [File/Data] : LIST, NLST, RETR, STOR, APPE, STOU, DELE, RNFR, RNTD, ABOR [Info/Meta] : STAT, SIZE, MDTM, HASH ===== ftp> USER admin 331 User name okay, need password. ftp> PASS 123456 230 User logged in, proceed. </pre>
F	Fault-injection or retransmission statistics	<pre> ❶ quile@LAPTOP-NIMBN78N:~/hybrid-ftp-system\$ export RDT_LOSS_RATE=0.10 ❷ quile@LAPTOP-NIMBN78N:~/hybrid-ftp-system\$ export RDT_CORRUPTION_RATE=0.10 ❸ quile@LAPTOP-NIMBN78N:~/hybrid-ftp-system\$ python3 server/tcp_control/tcp_server.py [2026-07-25 08:23:50] [INFO] [SERVER] TCP Control Engine listening on port 2121... [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:45666 - NONE IDLE [2026-07-25 08:23:56] [INFO] [FD 4] Session initialized for 127.0.0.1:45666 [2026-07-25 08:24:00] [INFO] [FD 4] Command received: USER admin [2026-07-25 08:24:03] [INFO] [FD 4] Command received: PASS 123456 [2026-07-25 08:24:03] [INFO] [FD 4] User 'admin' authenticated successfully. [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:45666 admin NONE IDLE [2026-07-25 08:24:14] [INFO] [FD 4] Command received: PASV [2026-07-25 08:24:14] [INFO] [FD 4] Command received: STOR bigfile.bin SIZE=1048576 SHA256=7d9bb0b41380d7a7e940a693db06128b5a83f13073505495d68739a103e2066 [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:45666 admin PASSIVE ACTIVE [ACTIVE SESSIONS] FD CLIENT USER MODE TRANSFER 4 127.0.0.1:45666 admin NONE IDLE </pre>

	<pre> quille@LAPTOP-N1PBN78N:~/hybrid-ftp-system\$ python3 client/tcp_client/control_client.py [Client] Connecting to 127.0.0.1:2121... 220 Welcome to RDT-FTP Server (Hybrid TCP/UDP) ===== HYBRID FTP CLIENT CLI - READY FOR COMMANDS ===== [Auth/Session]: USER, PASS, QUIT [Data Mode] : PASV, PORT, TYPE [Directory] : PWD, CWD, CDUP, MKD, RMD [File/Data] : LIST, NLST, RETR, STOR, APPE, STOU, DELE, RNFR, RNTD, ABOR [Info/Meta] : STAT, SIZE, MDTM, HASH ===== ftp> USER admin 331 User name okay, need password. ftp> PASS 123456 230 User logged in, proceed. ftp> STOR bigfile.bin ftp> 227 Entering Passive Mode (127,0,0,1,143,108) 150 TID=12740532324704879355 SIZE=1048576 SHA256=7d9bb0b41380d7a7e940a693db806128b5a83f13073505495d6 8739a103e2066 [RD] 100% 1048576/1048576 bytes retries=266 137.203s 226 Transfer complete. </pre>
--	---

7.4 Clean-Machine Demo Procedure

```

Terminal 1:
python server/tcp_control/tcp_server.py

Terminal 2:
python client/tcp_client/control_client.py

USER admin
PASS 123456
STOR sample.bin
PORT
RETR sample.bin
NLST
QUIT

```

Fault-injection demo (PowerShell): set `RDT_LOSS_RATE=0.10`, `RDT_CORRUPTION_RATE=0.05`, and `RDT_DUPLICATE_RATE=0.05` before starting the server and client. The CLI prints progress, retry count, and duration.

Place `sample.bin` in `storage/client_files` before starting the client. The client automatically enters Passive mode when no mode has been selected. Enter `PORT` explicitly to demonstrate Active mode.

Conclusion

The implementation satisfies the central Hybrid FTP requirement: a reliable application-layer data protocol built directly over UDP while TCP remains responsible for control and session state. Stop-and-Wait provides bounded flow, deterministic ordering, and straightforward recovery. CRC32 protects each datagram; SHA-256 protects the complete transferred object. The modular boundary between FTP and RDT allows the protocol to be tested independently and integrated in either data direction.

The primary performance limitation is the one-packet window. Throughput is approximately $\text{payload_size} / \text{RTT}$ when no loss occurs. A future extension could implement Go-Back-N or Selective

Repeat with adaptive windows, while preserving the existing packet codec, transfer ID, checksum, and integrity workflow.